

Reviewer Pack — Non-Commutative Rank Gap in Read-Twice BPs for IP_2

Entry 66b876365999 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 14:27:23 UTC

Non-Commutative Rank Gap in Read-Twice BPs for IP_2

Entry ID: 66b876365999 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 14:27:23 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP P computing the inner product mod 2 function on n variables, the non-commutative rank of its transition matrix is $\Omega(n)$, while for read-once BPs, it is $O(\log n)$.

Rationale (proposer's reasoning):

Free probability's non-commutative rank captures structural dependencies that differ between read-once and read-twice BPs, potentially exposing exponential gaps in size.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 259e5f26d98d2a3e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        factor = Fraction(1, A[i][i])
        for k in range(i+1, n):
            A[k][i] *= factor

        # Eliminate above
        for k in range(i):
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]

    return sum(abs(A[i][i]) for i in range(n))

def generate_read_once_bp(n):
    bp = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            bp[i][j] = random.choice([0, 1])
            bp[j][i] = bp[i][j]
    return bp

def generate_read_twice_bp(n):
    bp = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            bp[i][j] = random.choice([0, 1])
            bp[j][i] = random.choice([0, 1])
    return bp

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    read_once_bp = generate_read_once_bp(n)
    read_twice_bp = generate_read_twice_bp(n)

    read_once_rank = gaussian_elimination(read_once_bp)
    read_twice_rank = gaussian_elimination(read_twice_bp)

    results.append({
        "n": n,
        "read_once_rank": read_once_rank,
        "read_twice_rank": read_twice_rank
    })

mean_read_once_rank = sum(result["read_once_rank"] for result in results) / len(results)
mean_read_twice_rank = sum(result["read_twice_rank"] for result in results) / len(results)

conjecture_holds = all(mean_read_twice_rank <= 2 * math.log(n) and mean_read_once_rank >= n for n in n_values)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Non-Commutative Rank Gap",
    "metric_value": (mean_read_twice_rank, mean_read_once_rank),
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_read_twice_rank = sum(res["metric_value"][0] for res in results) / len(results)
    mean_read_once_rank = sum(res["metric_value"][1] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean_read_twice_rank={mean_read_twice_rank} mean_read_once_rank={mean_read_once_rank}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b702fda.py", line 96, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b702fda.py", line 67, in run_trial
    read_once_rank = gaussian_elimination(read_once_bp)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b702fda.py", line 29, in gaussian_elim
    factor = Fraction(1, A[i][i])
            ^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
```

ZeroDivisionError: Fraction(1, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing result collection. Support condition cannot be evaluated without valid data. | next: Fix Gaussian elimination to handle singular matrices (e.g., add checks for zero pivots or use symbolic computation)

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109480
2	propose	ollama_remote	qwen3:8b	0	0	79822
3	preregistration	ollama_remote	qwen3:8b	0	0	24101
4	novelty	ollama_remote	qwen3:8b	0	0	20721
5	novelty	ollama_remote	qwen3:8b	0	0	16177
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12670
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
8	judge	ollama_remote	qwen3:8b	0	0	16539

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 290125 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/66b876365999.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/66b876365999.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/66b876365999.tar.gz` (if generated)